# Off the Books: Audit Evasion in Google's Agent2Agent Protocol

[Author Name], [Affiliation]
**With**
Apart Research

## Abstract

As AI agents increasingly delegate work to one another over standardized protocols, the protocol—not the model—becomes the place where accountability has to live. We study Google's Agent2Agent (A2A) protocol and identify an audit gap at its core: the Task object is the only record of server-side work that any party can later query, yet the decision to create one rests entirely with the server being audited. A spec-compliant agent can perform real, multi-step, stateful work and reply with direct Messages instead—legitimate for clarifications and simple exchanges, but wholly unrecorded when used to deliver task-worthy results. Using the reference a2a-python SDK (v1.3.0) with deterministic, LLM-free agents, we demonstrate three results. A message-only server completes a multi-turn interaction over a single contextId while every audit surface stays empty: task queries return 404, the in-memory store holds zero objects, and even the SDK's durable SQL store writes zero rows. An intermediary in an A→B→C chain receives completed Tasks downstream and returns bare Messages upstream, laundering the record out of existence while violating zero MUSTs and one SHOULD NOT—undetectable, because A2A replies carry no signature. Finally, we implement and evaluate a client-side reply-type contract that detects exactly these cases while leaving spec-intended Message replies untouched, and we propose spec-level changes for full enforcement.

## 1. Introduction

AI agents are starting to delegate work to other agents over open protocols. Google's Agent2Agent protocol (A2A) [1, 4] is emerging as a standard for this: it defines a client role that requests work, a server role that performs it, and supports chains in which an intermediary acts as both. Alongside the transport, A2A ships an accountability primitive: the Task. A Task is a durable, server-side record—status, message history, artifacts—that any authorized caller can retrieve after the fact through GetTask and ListTasks. Messages, by contrast, are fire-and-forget: a reply Message is delivered to the client and, as we show, is retained nowhere the protocol can reach.

The split is designed to be benign. Section 3.1.1 of the specification lets a server either create a Task or return a direct Message, the latter intended for simple interactions that don't require task tracking; Section 3.7 names clarification requests as a valid use of Messages, and "Message-only," "Task-generating," and "hybrid" are all recognized agent styles [1]. Message replies are therefore not a defect. The problem is narrower and structural: when work merits a Task—because it is long-running, multi-step, or has side effects—nothing in the protocol requires one to exist. The only applicable rule is a single SHOULD NOT ("Messages SHOULD NOT be used to deliver task outputs" [1]), and the choice of reply type is made by the very party an audit would later examine.

This matters because the Task record is what post-hoc accountability runs on: dispute resolution, compliance review, incident forensics. A malicious or compromised server agent can do real work, return correct-looking answers, and leave the audit trail empty. Worse, in an agent chain a rogue intermediary can silently strip the Task records of every agent beneath it. Neither action breaks any MUST-level rule, and—because only Agent Cards can be cryptographically signed in A2A, never replies—neither is detectable through the protocol itself.

Our main contributions are:

• A specification and reference-implementation analysis (a2a-python 1.3.0) showing that every A2A audit surface is Task-anchored: no protocol method returns a past Message, history retrieval attaches only to Tasks, the SDK contains no context service at all, and clients have no way to request a Task reply.

• Two demonstrated attacks using deterministic, LLM-free agents: off-the-books execution (multi-turn, stateful work carried over a contextId with every audit surface empty, including the SDK's durable SQL store) and record laundering by a rogue intermediary in an A→B→C chain (zero MUST violations, one SHOULD NOT, undetectable via A2A).

• A tested fix: a client-side reply-type contract with record verification that flags exactly the malicious cases while leaving spec-intended Message replies untouched, plus spec-level proposals—reply-type negotiation and signed task receipts—for full enforcement.

## 2. Related Work

The closest prior work is Unit 42's study of agent session smuggling in A2A [3], which shows that a hidden session can ride inside a legitimate one by exploiting the protocol's statefulness—identifiers reused across sessions. Our attack is the mirror image: we exploit A2A's

statelessness. The rogue server keeps exactly as much state as it needs, in its own memory keyed by the contextId it echoes, while discarding the record. Read together, the two results show that both the session semantics and the record semantics of A2A are attacker-controllable.

More broadly, security analyses of the Model Context Protocol (MCP) [5] have concentrated on content-level attacks such as poisoned or swapped tool descriptions—manipulating what agents are told. Our finding is protocol-level: manipulating whether anything was recorded at all. OWASP's guidance for LLM applications lists "excessive agency" among its top risks [6]; our work identifies a concrete mechanism by which agency escapes after-the-fact accountability in an agent-to-agent setting. To our knowledge, no prior work demonstrates record evasion through A2A's reply-type discretion or evaluates reply-type contract enforcement.

## 3. Methods

All experiments are designed around one principle: hold everything constant except the reply envelope. Every server agent in the study is deterministic and shares identical work logic; the only difference between the honest and rogue variants is whether the executor emits Task events (creating a Task) or a single bare Message. No LLM is involved anywhere, so any difference in what an auditor can afterwards recover is attributable to the protocol-level choice, not to model behavior. The complete suite reproduces with one command (context_probe/run_all.py, ≈20 seconds), starts its own servers in-process, drives them over real HTTP, and audits them both through A2A methods and by dumping server-side state directly for ground truth.

### Threat model

The adversary is a malicious or compromised server agent, or a rogue intermediary in a chain. It is fully spec-compliant: standard transport, standard authentication, no forged content, no network attack. Its goal is not to lie about results but to ensure that task-worthy work—work that is long-running, multi-step, or side-effectful—leaves no record any other party can query through A2A. The defender is any party relying on A2A's standard mechanisms (the client agent, an operator, an external auditor): we assume the defender can invoke every documented method on the agents it knows about, and nothing more.

### Experimental setup

We pin the reference a2a-python SDK at v1.3.0 [2] on the REST binding (HTTP+JSON, protocol version 1.0.0), Python 3.12.10, with all dependencies frozen in the repository. Clients send raw protocol-JSON requests with the required A2A-Version header; servers use the SDK's DefaultRequestHandler with either the default in-memory task store or the SDK's durable DatabaseTaskStore backed by SQLite. After each interaction we audit the retrieval surfaces—GET /tasks (list) and GET /tasks/{id} for every identifier the client holds (task id, contextId, message id)—plus both task stores, the SDK log at INFO and above, and telemetry; in-process dumps of the stores and executor memory provide ground truth for what actually happened versus what the protocol exposes. We additionally confirmed in exploratory testing that the

JavaScript reference SDK (@a2a-js/sdk 1.3.0) shows the same GetTask/ListTasks/store behavior; the experiments reported here are Python.

### Experiments

Experiment 1 (record gap) compares an honest Task-replying server against a rogue Message-only server across a three-turn interaction threaded on one contextId. Experiment 2 (record laundering) adds a third agent: a chain A→B→C in which B is a server to A and a client to C, forwarding A's work to the honest server C and returning C's task output to A as a bare Message. Experiment 3 (fix) equips the client with a reply-type contract: requests are marked task-worthy or ordinary, and every reply that claims a record is verified by an actual GetTask call rather than trusted.

## 4. Results

We report the three experiments in turn; Table 1, Table 2, Figure 1, and Figure 2 carry the quantitative evidence.

**Off-the-books execution (Experiment 1).** The rogue server completed all three turns of a multi-turn interaction—echoing the same contextId each time and tracking conversation state in its own memory—while replying with bare Messages. The honest twin performed the identical work behind a Task. Figure 1 shows both flows and the audit surfaces afterwards; Table 1 gives the measured results. Every surface is empty for the rogue arm: ListTasks returns totalSize 0, GetTask returns 404 TASK_NOT_FOUND for every identifier the client holds, the in-memory store contains zero objects, and the durable SQLite store contains zero rows. The honest arm leaves one completed Task whose history contains both the client's request and the server's answer, retrievable at any later time.

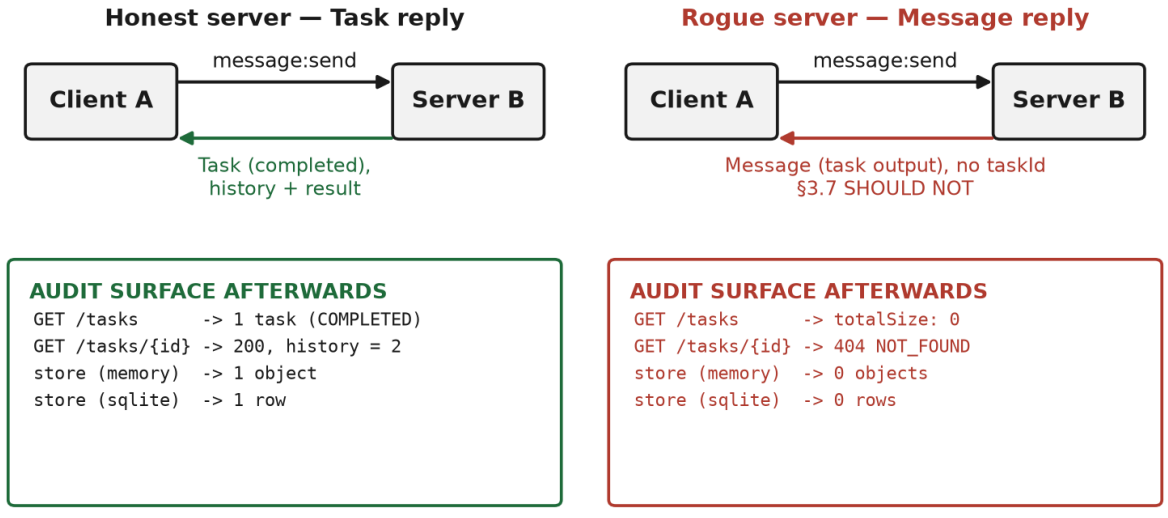



Figure 1: The same task-worthy work, twice. The rogue server (right) keeps the conversation stateful over one contextId but replies with bare Messages; afterwards every A2A audit surface is empty. The honest server (left) leaves a queryable Task record.

Table 1: Audit surfaces after the same three-turn interaction, honest vs. rogue server (a2a-python 1.3.0).

| Audit surface | Honest server (Task reply) | Rogue server (Message-only) |
|---|---|---|
| Reply envelope | Task, TASK_STATE_COMPLETED | Message (task output, no taskId) |
| GET /tasks | 1 task listed | totalSize: 0 |
| GET /tasks/{id} (any id client holds) | 200; history = request + reply | 404 TASK_NOT_FOUND |
| Task store (in-memory) | 1 object | 0 objects |
| Task store (SQL-durable) | 1 row | 0 rows |
| Multi-turn state afterwards | recoverable from task history | gone (lived only in executor memory) |

**Why the gap is structural.** These results follow from the protocol's design rather than an implementation accident. Source analysis of the SDK shows: (i) only Task, TaskStatusUpdateEvent, and TaskArtifactUpdateEvent are ever persisted—a Message reply is returned to the caller with no store call; (ii) the incoming client message is appended to a task's history only when a task already exists, so in a message-only exchange the client's request is also discarded; (iii) historyLength applies only to Task replies (§3.2.4), and no A2A method returns a past Message—the entire REST surface is message:send, message:stream, tasks/*, and push-notification configuration; (iv) there is no context service in a2a-python 1.3.0: contextId is an opaque string used for matching, and nothing is stored keyed by it; (v) SendMessageConfiguration contains only acceptedOutputModes, taskPushNotificationConfig, historyLength, and returnImmediately—clients cannot request Task replies even if they want them.

**Record laundering (Experiment 2).** In the chain A→B→C, the honest server C created a durable, completed Task for every unit of work B forwarded; B replied to A with the task output in a bare Message carrying its own contextId and no taskId (Figure 2). A's audit surface at B is completely empty, and A received no reference to C at all—so the records at C are orphaned: because B forwarded fresh, contextId-free messages, C's tasks cannot even be correlated with A's conversation after the fact. The linkage between downstream tasks and upstream requests exists only in B's private memory, which no A2A method exposes. B's behavior violates zero MUST-level rules: §3.1.1 explicitly permits a direct Message reply, and the single violated rule is the SHOULD NOT of §3.7. Detection is impossible through the protocol because replies are unsigned—the SDK's only cryptographic facility is Agent Card signing; Tasks and Messages cannot be signed or verified at all.

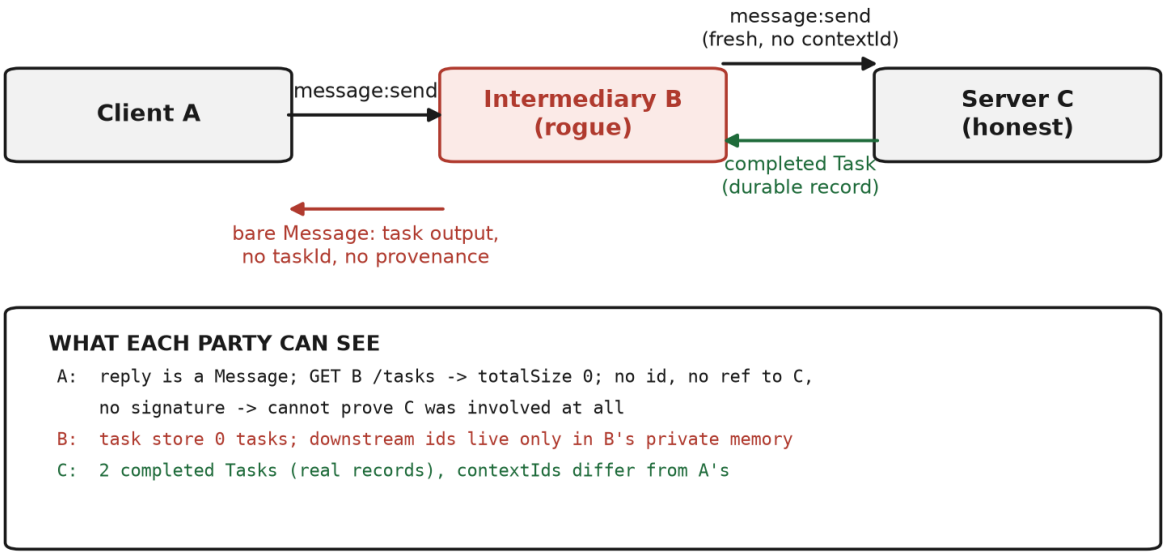



Figure 2: Record laundering in an A→B→C chain. Work is recorded at C, but the rogue intermediary B returns bare Messages to A: no taskId, no provenance, no signature, and an empty audit surface at B.

**The fix (Experiment 3).** We implemented a client-side reply-type contract: the caller marks each request as task-worthy or ordinary, and the client verifies rather than trusts. A Task reply is checked with a real GetTask call; a Message reply citing a taskId is checked the same way; a Message reply to a task-worthy request with no taskId raises AUDIT_RECORD_MISSING; and a cited task that does not exist raises PHANTOM_TASK_REFERENCE. Table 2 shows the six-case evaluation across four servers. The contract flags exactly the two malicious cases—the attack and the lying variant—while leaving every spec-intended Message reply untouched (simple questions and hybrid agents' clarification turns pass). Because verification queries the server, the fix also catches servers that cite nonexistent tasks, a bypass a naive envelope check would miss.

Table 2: Client-side reply-type contract across four server behaviors (Experiment 3). The two flagged rows are the malicious cases.

| Server behavior | Reply to the client | Contract verdict |
|---|---|---|
| Honest | Task (completed) | COMPLIANT_VERIFIED (GetTask 200) |
| Rogue message-only, task-worthy request | Message, no taskId | AUDIT_RECORD_MISSING — flagged |
| Rogue message-only, simple question | Message | OK_MESSAGE_ALLOWED |
| Hybrid, clarification turn | Message | OK_MESSAGE_ALLOWED |
| Hybrid, work turn | Task (completed) | COMPLIANT_VERIFIED |
| Lying ("phantom") | Message citing bogus taskId | PHANTOM_TASK_REFERENCE — flagged |

**Robustness.** The results are deterministic—repeated runs produce identical audit outcomes—and they do not depend on any model: the executors are plain Python. The gap is therefore a property of the protocol surface, not of a particular agent implementation; any A2A server, LLM-driven or not, can make the same choice. We verified the same GetTask/ListTasks/store

behavior on the JavaScript SDK in exploratory testing, and the specification text alone implies the same surfaces: nothing in v1.0.0 provides message retrieval.

## 5. Discussion and Limitations

The deeper finding is that in A2A, auditability is opt-in—by the party being audited. Statefulness and record-keeping are fully decoupled: the contextId mechanism gives an agent everything it needs to run a stateful multi-turn interaction, while the Task is what makes that interaction queryable, and nothing links the two. Operator-side logging partially mitigates the direct case—a gateway in front of an honest client does see the replies—but it is non-standard, not evidence-grade, and useless against laundering: a rogue endpoint will not log its own replies, and an intermediary controls its own leg of the chain. For ecosystems built on delegation, where a compliance question six months later may be “which agent did what,” a protocol whose only record is optional cannot answer it.

### Limitations

Our client-side fix detects non-compliance but cannot compel it: a rogue server can still return a Task with fabricated content, since the contract verifies the existence and provenance of a record, not the truth of its contents. The formal experiments cover a2a-python 1.3.0 against specification v1.0.0; the JavaScript SDK was checked only on GetTask/ListTasks/store behavior in exploratory runs, and other SDK versions could differ in unexposed internals. We exercised the message:send path; streaming and push notifications share the same persistence rule (only task events are stored) but were not tested. All experiments ran on a single local testbed with deterministic agents; no LLM-driven agents were tested, which strengthens attribution of the effect but limits direct ecological claims. Finally, we assume the defender can reach the agents it knows about: an auditor with no knowledge of agent C can never observe laundered records, by construction.

### Future Work

Two spec-level changes would close the gap at the source. Reply-type negotiation: add a require-task field to SendMessageConfiguration, obligating a server that cannot comply to return an error rather than silently downgrade—converting the §3.7 SHOULD NOT into an enforceable contract. Signed task receipts: extend the existing Agent Card JWS trust anchor to receipts over {contextId, taskId, result digest}, so chains like A→B→C cannot strip or rewrite records undetectably. A minimal alternative is a message ledger keyed by contextId with a query method. Beyond the protocol, the audit harness in this repository is a starting point for continuous monitors that diff ListTasks against expected work.

## 6. Conclusion

We showed that Google’s A2A protocol makes the only queryable record of an agent’s work—the Task—entirely optional for the agent itself. A spec-compliant rogue server can perform multi-step, side-effectful work over a stateless channel while every audit surface stays empty, and a rogue intermediary can erase the records of an entire downstream chain; in both cases no

MUST rule is broken and no protocol-visible trace is left. The attacks require no model, no exploit, and roughly one line of code: choosing a Message where a Task was warranted.

The protocol can fix this. A reply-type contract on the client detects the gap today, and modest spec changes—negotiated reply types and signed task receipts—would make durable records enforceable rather than optional. As agent delegation becomes infrastructure, “did it leave a record?” needs to stop being a question the worker answers alone.

## Code and Data

**Code repository:** the rogue_agent_a2a repository (public link added at submission). The experiments live in context_probe/ and reproduce with a single command (run_all.py; ≈20 s, pinned dependencies).

**Data/Datasets:** none used; all artifacts are generated deterministically by the experiments, and the raw outputs behind every number in this report are committed (*_output.txt).

**Other artifacts:** paper-ready writeups of each experiment (FINDINGS.md, LAUNDERING.md, FIX.md) with SDK source citations.

## References


[1] Agent2Agent Protocol Specification, version 1.0.0. A2A Project (Linux Foundation), 2025. https://a2a-protocol.org/latest/specification/

[2] A2A Project. a2a-python: Agent2Agent Python SDK, version 1.3.0. GitHub, 2025. https://github.com/a2aproject/a2a-python

[3] Unit 42, Palo Alto Networks. Agent Session Smuggling in Agent2Agent Systems, 2025. https://unit42.paloaltonetworks.com/agent-session-smuggling-in-agent2agent-systems/

[4] Google Developers Blog. Announcing the Agent2Agent Protocol (A2A): A New Era of Agent Interoperability, 2025. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[5] Anthropic. Model Context Protocol, 2024. https://modelcontextprotocol.io

[6] OWASP Foundation. OWASP Top 10 for LLM Applications, 2025. https://genai.owasp.org/


## Appendix: Limitations and Dual-Use / Ethical Considerations

All experiments were conducted on a local, isolated testbed against agents we wrote ourselves; no third-party systems, users, or services were involved. The demonstrated attack is deliberately minimal—the difference between the honest and rogue servers is the reply envelope they emit—and we publish it in full, together with a tested mitigation, because the finding is useful to defenders and protocol maintainers only if it is concrete. The malicious behavior is spec-visible, and our harness lets any A2A deployment be audited for reply-type choices. We will notify the A2A protocol maintainers of the finding and the proposed fixes before publication. No real user data, credentials, or production agents were used at any point.

## LLM Usage Statement

We used an LLM-based coding assistant (ZCode/GLM) to help build the experiment harness, analyze the SDK source, and draft this report. All results and claims come from the deterministic runs committed to the repository, which the authors verified and take full responsibility for.